PHAEMA Technical Manual

Nov 13, 2026 · @Stuart

About this manual

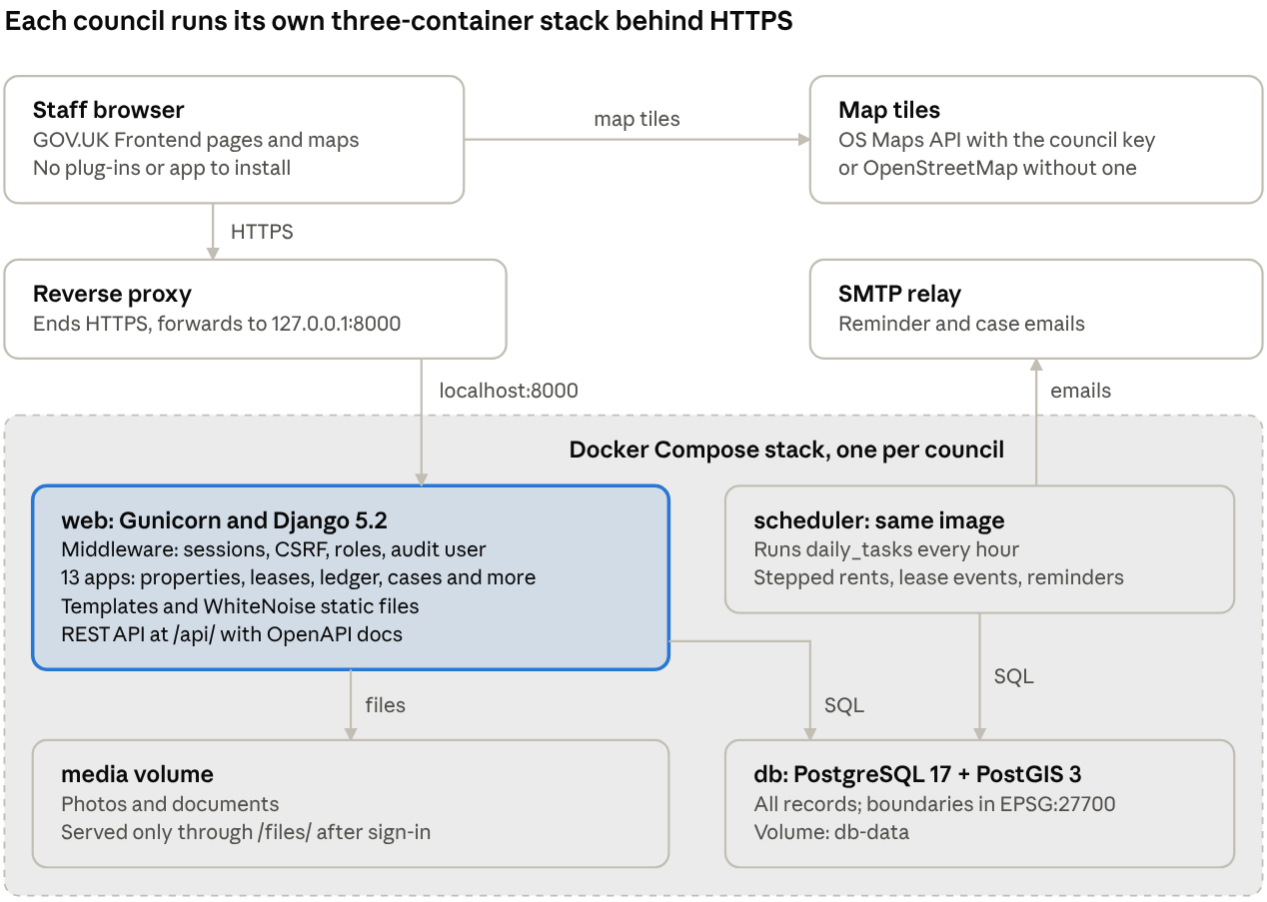
This manual explains how PHAEMA is built, installed, secured and run, so a council's IT team or a supplier can assess, host and support it. It covers the code on the `main` branch of phaema/phaema-app as of November 2026.

Reader	Read first
Council IT and security teams	Architecture, Security, Compliance mapping
Hosting suppliers and system administrators	Installation, Deployment and operations, Troubleshooting
Developers and contributors	Technology stack, Code layout, Data model, Testing, Contributing

The user manual covers how staff use the system. This manual covers the software underneath it. PHAEMA is free software under the GNU Affero General Public License v3.0 or later (AGPL-3.0-or-later).

Architecture overview

PHAEMA is a single server-rendered Django application with a PostgreSQL/PostGIS database and an hourly scheduler, packaged as three Docker containers that each council runs on its own.



PHAEMA architecture · one stack per council

Only the reverse proxy faces the network; the `web` container listens on localhost. The scheduler uses the same image and database as `web` and also shares the `media` volume. The browser fetches map tiles directly from Ordnance Survey or OpenStreetMap.

How a request is handled

1. The browser sends an HTTPS request; the proxy ends TLS and forwards it to Gunicorn on port 8000.
2. Django middleware runs in order: security headers, WhiteNoise (static files are answered here), sessions, language, CSRF, authentication, messages, clickjacking, then PHAEMA's `CurrentUserMiddleware` (records who is acting, for the audit trail) and `RoleRequiredMiddleware` (blocks users with no role).
3. The URL is routed to an app's view, which checks the user's role and, for changes, `can_edit`.
4. The view reads or writes models through the Django ORM; every save or delete of an audited model writes an `AuditEntry`.
5. A Django template renders GOV.UK Frontend HTML, or the view returns a file (Excel, CSV, Word, GeoJSON, iCalendar) or JSON for the API.

Technology stack

PHAEMA is a Python 3.12 Django 5.2 application on PostgreSQL 17 with PostGIS, rendered on the server with GOV.UK Frontend. Every component is open source and none needs a paid licence. Version ranges come from `requirements.txt` and the Dockerfiles; the image installs the newest release within each range.

Server side

Component	Version	Licence	What PHAEMA uses it for
Python	3.12 (image `python:3.12-slim-bookworm`)	PSF	Language and runtime
Django	5.2 LTS (`>=5.2,<5.3`)	BSD-3-Clause	Web framework: models, ORM, migrations, forms, templates, authentication, admin, translation
GeoDjango (`django.contrib.gis`)	with Django	BSD-3-Clause	Spatial fields and queries for property boundaries and points
GDAL	Debian bookworm `gdal-bin`	MIT	Reads INSPIRE GML and other spatial formats for GeoDjango
Django REST framework	`>=3.15,<4`	BSD-3-Clause	The open JSON API at `/api/`
drf-spectacular	`>=0.28,<1`	BSD-3-Clause	OpenAPI 3 schema at `/api/schema/` and interactive docs at `/api/docs/`
django-environ	`>=0.12,<1`	MIT	Reads settings from environment variables and `.env`
psycopg 3	`>=3.2,<4` (binary)	LGPL-3.0	PostgreSQL driver

Component	Version	Licence	What PHAEMA uses it for
Gunicorn	`>=23,<24`	MIT	Production WSGI server, 3 workers, 300 s timeout
WhiteNoise	`>=6.8,<7`	MIT	Serves compressed, cache-busted static files from the app
openpyxl	`>=3.1,<4`	MIT	Excel import templates, imports and every Excel export
Pillow	`>=11,<13`	MIT-CMU (HPND)	Property photo handling
Python-Markdown	`>=3.7,<4`	BSD-3-Clause	Renders the in-app help pages
python-docx	`>=1.1,<2`	MIT	Word export of letters and notices from the template library
extract-msg	`>=0.48,<1`	GPL-3.0 (compatible with AGPL)	Reads Outlook `.msg` emails filed on cases

Database

Component	Version	Licence	Use
PostgreSQL	17 (image `postgres:17-bookworm`)	PostgreSQL Licence	All application data
PostGIS	3 (`postgresql-17-postgis-3`)	GPL-2.0	Spatial types, indexes and functions

The automated test run on GitHub uses `postgis/postgis:16-3.4`, so PHAEMA is tested on PostgreSQL 16 and run on 17.

Browser side

Component	Version	Licence	Use
GOV.UK Frontend	5.14.0, vendored	MIT, © Crown copyright	Accessible page components and layout. The GDS Transport font is removed (it is licensed only for GOV.UK) and no GOV.UK branding is used
OpenLayers	10.11.0, vendored	BSD-2-Clause	Property maps and boundary drawing
Atkinson Hyperlegible	vendored	SIL OFL 1.1	Typeface for the Clear style
PDF.js	6.5.438, vendored	Apache-2.0, © Mozilla Foundation	Shows PDFs inside PHAEMA. Its bundled fonts and image decoders keep their own open-source licences beside them
PHAEMA's own scripts	`static/js/phaema.js`, `phaema-map.js`	AGPL-3.0	Collapsible panels, map set-up and the PDF viewer page (pdf-view.mjs); no framework, no build step

There is no Node.js build, no content delivery network and no third-party tracking. All browser files are served from the PHAEMA server. The one optional outside call is for map tiles: Ordnance Survey Maps API tiles when a council enters its OS key, otherwise OpenStreetMap tiles.

Development and testing

Tool	Version	Use
pytest + pytest-django	`>=8,<9`, `>=4.9,<5`	Automated tests

Tool	Version	Use
pyflakes	any	Lint for unused imports and undefined names
GitHub Actions	`ubuntu-24.04`	Runs migrations check and tests on every pull request
Docker and Docker Compose	current	Local and single-server running

Code layout

The repository is one Django project (`phaema/`) with fourteen apps under `apps/` , one per business area. Each app keeps its own models, views, URLs, migrations and tests.

Top-level folders

Path	Contents
`phaema/`	Project settings (`settings.py`), root URLs and the WSGI entry point
`apps/`	The Django apps listed below
`templates/`	Shared and per-app Django templates (GOV.UK Frontend markup)
`static/`	`css/` (`phaema.css` , `themes.css` , `skins.css`), `js/` , `vendor/` , `help/` screenshots, `training/` e-learning
`training/`	Source of the e-learning courses (`course.py`) and the script that builds them
`website/`	Source and build script for the public project website
`docs/`	Roadmap, roles and Mac set-up notes
`docker/db/`	The PostgreSQL 17 + PostGIS image
`Dockerfile` , `docker-compose.yml` , `phaema.sh`	Container build, the three-service stack and the control script
`.github/workflows/tests.yml`	The automated test run

Path	Contents
`LICENSE`, `LICENSING.md`, `CONTRIBUTING.md`	Licences and contribution rules

Django apps

App	Responsibility	Main models
`core`	Dashboards (estate and My work), roles, audit trail, settings, skins and themes, protected file serving	`ModuleSettings`, `AuditEntry`, `StaffProfile`
`properties`	Property register, hierarchy, photos, titles, INSPIRE boundaries, maps, open data, demo data	`Property`, `PropertyPhoto`, PropertyPlan, `Title`, `TitleDocument`, `InspireParcel`
`leases`	Leases and licences, parties, breaks, rent schedules, lease events, reminders, calendar feeds	`Lease`, `Party`, `BreakOption`, `RentScheduleEntry`, `LeaseEvent`, `ReminderLog`, `CalendarFeed`
`ledger`	Rent demands, VAT, receipts, arrears, statements, finance export	`DemandRun`, `Charge`, `Receipt`, `ExportBatch`
`servicecharges`	Service charge schedules, budgets, apportionment, year-end reconciliation	`Schedule`, `ScheduleMember`, `ScheduleYear`, `BudgetLine`, `Expenditure`, `CostHeading`
`cases`	Case types, stages and tasks, approvals, notes, correspondence, MEES, dilapidations and conflict checks	`CaseType`, `Case`, `CaseTask`, `CaseNote`, `Correspondence`, `CaseDocument`, `Approval`
`maintenance`	Contractors, contracts, components, planned tasks, reactive jobs, recharges	`Contractor`, `Contract`, `Component`, `PlannedTask`, `Job`
`compliance`	Compliance types, requirements, certificates, actions, building safety (Golden Thread tiers)	`Requirement`, `Certificate`, `Action`, `BuildingSafety`

App	Responsibility	Main models
`valuations`	CIPFA asset classes, valuation programme, Red Book fields, components, audit reviews	`AssetClass`, `Valuation`, `ValuationComponent`, `Valuer`, `AuditReview`
`projects`	Developments, acquisitions, disposals, programmes and reviews: gateways, budget by financial year, risks, linked work, completion checklist	`Project, ProjectType, GatewayDecision, BudgetLine, BudgetYear, Risk, CompletionStep, ProjectAccess`
`documents`	Letter and notice templates filled from cases, Word export	`DocumentTemplate`
`reports`	Report builder, saved reports, Excel and CSV export of any list	`Report`
`imports`	Spreadsheet templates and imports of properties, parties, leases, contractors and components	none
`help`	In-app help pages written in Markdown	none

Management commands

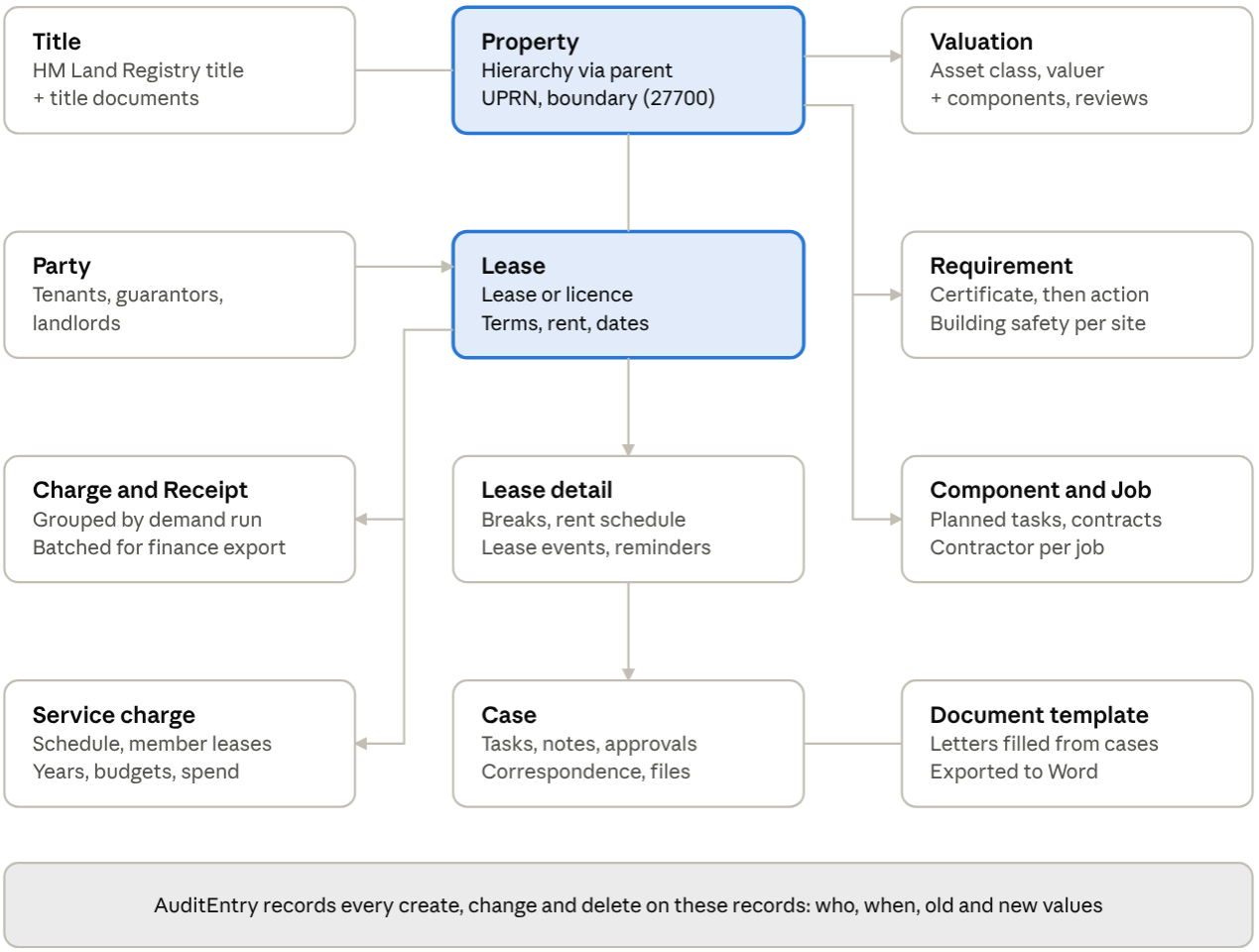
Command	Run by	Does
`daily_tasks`	The `scheduler` container, hourly	Stepped rents, lease events and reminder emails
`send_reminders`	`./phaema.sh reminders`	Sends lease event reminders that are due
`import_inspire`	`./phaema.sh import-inspire FILE`	Loads HM Land Registry INSPIRE parcels from a zip
`load_demo_data`	`./phaema.sh demo-data [--reset]`	Loads the fictional Exampleton and Northfield estates

Data model

PHAEMA holds 77 models (database tables) of business records across twelve apps, organised around the property and the lease. Every other record hangs off one of those two, and every change is audited.

Property and Lease sit at the centre of the data model

Arrows point from one record to its many; plain lines are many-to-many



Main records and how they link

Cases can also link directly to a property, party or lease event, and jobs to a lease or case; those secondary links are left off the picture.

Key design rules

- **Property hierarchy.** A property can be part of another (`parent`), so a site, building and unit form a tree. Deleting a parent that has children is blocked (`PROTECT`).
- **Identifiers.** `reference` is the council's own unique asset ID; `uprn` is an optional Unique Property Reference Number of up to 12 digits.

- **Spatial data.** Boundaries are `MultiPolygonField` in British National Grid (SRID 27700), with easting and northing for a point. INSPIRE parcels are stored separately (`InspireParcel`) and a surveyor picks parcels on the map to set a property's boundary from them. Geometry is converted to WGS 84 only for the browser.
- **Leases cover several premises.** `Lease.properties` is many-to-many, as titles are to properties.
- **Money is never deleted.** Charges, receipts and demand runs use `PROTECT`, and a credit note is a separate charge pointing at the one it credits. Invoice numbers are unique.
- **Single settings row.** `ModuleSettings` holds the council's switched-on modules and letter details.
- **Audit by signals.** `AuditEntry` uses Django's content types to point at any record, storing the changed fields as JSON.

Lease management and classification fields

Added in November 2026 (`leases.0005` , `properties.0007` and `properties.0008`). None of them adds a table except `Portfolio` .

Record	Fields	Used by
`Lease`	`time_of_essence`	`LeaseEvent.time_of_essence` (reviews only); `events.essence_summary()` for the dashboard; events list `?essence=1` ; My work
`Lease`	`demands_on_hold` , `demands_on_hold_reason`	`billing.billable_leases()` leaves these out and `billing.held_leases()` lists them in the demand run preview; `servicecharges/apportion.py` skips them too
`Lease`	`billing_contact_name` , `billing_email` , `billing_phone` , `billing_address`	`Lease.billing` returns each detail, falling back one by one to the counterparty's main contact; used by the rent account, statement, finance export and reports

Record	Fields	Used by
`Lease`	`arrears_referral` (debt collectors or legal), `arrears_referred_on`, `arrears_referral_reference`	Arrears list tag, `?referred=` filter and download
`LeaseEvent`	`estimated_rental_value`	Review and expiry events only (`takes_erv`); `erv_uplift` is the difference from the current rent
`LeaseDocument`	`review_date`; document type `insurance`	`review_rag()`; `events.documents_for_review()`; `DocumentListView` at `/leases/documents/`; report dataset `lease_documents`
`Property`	`portfolio` (to `Portfolio`, protected), `strategy_section`, `purpose_review_date`	Register filters, reports, import template, API

`events.event_summary()` returns the dashboard rows. Lease expiries get a total row and then one row for each 1954 Act status (`EXPIRY_SPLIT`), each with the query string for the events list (`?type=expiry&security=inside|out|other`). `events.tenancies_at_will()` counts live tenancies at will; the dashboard links to the lease list with `?agreement_type=taw&live=1`.

The case type `insurance_certificate` is seeded by `apps/cases/seed.py` on migrate, and `cases:new` accepts `?type=<code>` to preselect a case type. Tests: `apps/leases/test_lease_management.py` and `apps/properties/test_portfolios.py`.

Government Property Data Standard fields and export

Added on 13 November 2026 by `properties.0010_data_standard_fields`, after PHAEMA's property record was compared with the Government Property Data Standard (GovS004-PDS010, version 1.0).

- Two new tables. `PropertyUse` is the council's list of uses (`name`, `standard_usage`, `active`, `order`), seeded from `apps/properties/uses.py` and kept under Manage data. `PropertyCost` holds one row for each property and financial year (`year_starting`, `running_cost`, `rates`, `notes`), unique on property and year.
- New fields on `Property`: `use_type` (to `PropertyUse`, protected), `acquired_on`, `built_year`, `condition_grade` (A to D) and `condition_surveyed_on`, `listed_building`, `scheduled_monument`, `sssi`, `conservation_area` and

`designation_notes`, `vacant_since`, `vacancy_status`, `available_from` and `available_until`, and `disposal_method`, `disposal_reason`, `disposal_expected_on` and `disposal_expected_value`. Designations are blank for not known, so a gap can be told from a checked "no".

- `Property.save()` fills `built_year` from the first year in `built` when it is empty, and clears the vacancy fields when the occupation is not vacant. The migration does the first of these for existing records and matches existing use descriptions to the seeded list by name.
- `apps/properties/standard.py` builds the export. `gather(user)` returns plain rows for four sheets (properties, contracts held as tenant, vacant spaces, components), including valuations, energy certificates and components only where `apps.core.access.scope` gives that user the whole module. `workbook(data)` writes them with openpyxl, each column headed with the standard's field name and reference; `coverage(data)` gives the share of current records with each field recorded; `NOT_EXPORTED` lists the standard's fields left out, with the reason. Nothing is defaulted: an unrecorded fact is a blank cell.
- Views `properties:data-standard` (the page) and `properties:data-standard-download` (the workbook) are open to administrators and surveyors. Surveyors gain add, change and delete on `PropertyCost`.
- The field mapping was made from the standard's HTML page. It should be checked line by line against the published PDF before any public claim is made, and the wording to use is "mapped to", not "compliant with".
- Tests: `apps/properties/test_data_standard.py`.

The full field list for every model is in each app's `models.py`, and the Django admin at `/admin/` shows them all.

Front end

Every page is HTML built on the server by Django templates using GOV.UK Frontend components. Every task works without JavaScript; scripts only add conveniences on top.

Templates and styles

- `templates/base.html` sets the header, menus, skip link, language and the `data-skin`, theme and text-size attributes on `<html>`.
- Each app has its own template folder; shared pieces live in `templates/core/` (form fields, errors, hints, pagination, sortable table headings, download buttons, place filters).

- `static/css/phaema.css` holds PHAEMA's own layout; `themes.css` the colour themes; `skins.css` the skins.

Personal display settings

Each setting is a link that stores a cookie, so it works without JavaScript and follows the user on that browser.

Setting	URL	Cookie	Options
Text size	`/text-size/<size>/`	`phaema_text_size`	Smaller, Standard, Large, Largest
Colour theme	`/theme/<theme>/`	`phaema_theme`	Standard, Dark, High contrast, Match my device
Style (skin)	`/style/<skin>/`	`phaema_skin`	Classic, Clear (Atkinson Hyperlegible), Compact
Dashboard	`/?view=mine` or `estate`	`phaema_dashboard`	My work, Estate

JavaScript

- `static/js/phaema.js`: photo gallery, header menus (one open at a time, Escape closes), and collapsible panels. A panel is `section.phaema-panel[data-panel]`; the script adds a show/hide button to its heading and remembers closed panels in `localStorage` under `phaema_closed_panels`.
- `static/js/phaema-map.js`: sets up OpenLayers maps, shows boundaries and lets surveyors draw them. Boundaries are stored in British National Grid (EPSG:27700) and sent to the browser as GeoJSON in WGS 84 (EPSG:4326); the map itself displays in Web Mercator.
- No framework, bundler or Node.js build is used; files are served as written, with cache-busting names from WhiteNoise.

Accessibility

PHAEMA targets WCAG 2.2 level AA. GOV.UK Frontend supplies tested components. On top of that:

- every colour theme and skin was designed to keep text contrast at AA, with a High contrast theme for those who need more;
- collapsible panels use real buttons with `aria-expanded`, and closed content is hidden from screen readers as well as from view;

- maps always have a text alternative (reference, address and grid reference) and nothing depends on the map;
- motion is switched off when the device asks for reduced motion;
- every page works with a keyboard alone; pages are checked with axe-core during development, with no violations on the main pages. A formal accessibility audit and statement are still to be done.

Security

Access is controlled by four roles held as Django groups, with record-level rules for surveyors, and every change to business records is written to an audit trail that cannot be edited.

Authentication

- Users sign in with a username or an email address, and a password, at `/login/`. Passwords are stored with Django's default PBKDF2-SHA256 hashing.
- Password rules: at least 12 characters, not similar to the user's details, not a common password, not all numbers.
- Sessions use Django's signed session cookie; with `DEBUG` off, session and CSRF cookies are marked Secure unless `SECURE_COOKIES=False` (only for local testing over plain HTTP).
- The API accepts the same session or HTTP Basic authentication (Django REST framework defaults), so it must only be offered over HTTPS.
- Two-step sign-in (multi-factor authentication) uses any authenticator app (TOTP, RFC 6238) with single-use recovery codes. A council setting requires it for administrators (the default), for everyone, or makes it optional. The admin site signs in through the same page, and the API refuses a password alone for accounts that use it. Optional single sign-on with the council's own accounts is described below.

Email sign-in is `UsernameOrEmailBackend` in `apps/core/auth_backends.py`, the first entry in `AUTHENTICATION_BACKENDS`. It extends Django's `ModelBackend`, so permissions are unchanged. The username is tried first; an email address is matched without regard to case and only when exactly one account has it, because Django does not make addresses unique. An unknown name still runs the password hasher, so timing does not show whether an account exists. Two-step sign-in and the API rule apply the same way whichever name is typed. Tests: `apps/core/test_email_sign_in.py`.

Forgotten-password reset is in `apps/core/password_reset.py`. The form at `/account/forgot-password/` takes an email address or a username and always shows the same "check your email" page, so it does not reveal whether an account exists. An

email goes only to active accounts that have an email address: the account with that username, or every account with that address, each with its own link. The link (`/account/reset-password/<uid>/<token>/`) uses Django's `PasswordResetConfirmView` with `ResetTokenGenerator`, a subclass of Django's token generator with its own salt and a time limit of `PHAEMA_PASSWORD_RESET_MINUTES` (60 by default). The token is tied to the password hash and last sign-in, so it works once. It is separate from the first-password link sent to a new test account, which lasts the days set by Django's `PASSWORD_RESET_TIMEOUT`; neither kind of link opens the other page. Requests are limited through the cache to five an hour from one connection and three emails an hour to one account. Choosing a password does not sign the person in, so two-step sign-in still applies, and Django signs out their other sessions. When `EMAIL_HOST` is blank the form is replaced by a message to ask an administrator. On a test site, usernames starting `demo.` cannot be reset. Tests: `apps/core/test_password_reset.py`.

Single sign-on is the first connector, in the optional app `apps/connectors`. It uses OpenID Connect, so Microsoft Entra ID (Microsoft 365), Keycloak and other services are only different settings. It is off unless the installer sets `PHAEMA_SSO_ISSUER`, `PHAEMA_SSO_CLIENT_ID` and `PHAEMA_SSO_CLIENT_SECRET` in the server's `.env` and an administrator ticks **Sign in with the council's own accounts** in Module settings; with either missing, `/login/sso/` returns 404 and PHAEMA behaves as before. `sso.py` runs the authorisation code flow with PKCE: `/login/sso/` keeps a state, nonce and code verifier in the session and redirects to the service; `/login/sso/callback/` checks the state, exchanges the code at the token endpoint and checks the ID token's signature (keys from the service's `jwks_uri`, read with PyJWT), issuer, audience, expiry and nonce. The service's published settings are read from `/.well-known/openid-configuration` and cached for an hour. PHAEMA asks only for the `openid profile email` scopes and stores no tokens.

A person is matched by `SsoIdentity` (issuer and subject) once linked; the first time, by email address to exactly one active account that is not a superuser and not already linked. If the council allows it (`sso_create_accounts`), an account with no usable password is created instead. `SsoGroupMapping` rows give a group or app role from the `groups` and `roles` claims a PHAEMA role, an access area or both; `sync_groups()` applies them at each sign-in, changing only roles and areas that some mapping names, and leaving everything alone when the service reports more groups than it can list. The session is marked with `SsoBackend`, which never checks a password; `TwoStepRequiredMiddleware` does not ask those sessions for PHAEMA's two-step sign-in. `password_allowed()` applies the council's rule on PHAEMA passwords (`sso_passwords`): by default a linked person's password is refused by `UsernameOrEmailBackend` and they are sent no reset email, while administrators and superusers keep theirs as a fallback. Links, account

creation and role changes go to the change log. The set-up guide for a council's IT team is `docs/single-sign-on.md`. Tests, against a stand-in service with its own signing key: `apps/connectors/test_sso.py`.

Roles and permissions

`apps/core/roles.py` creates or refreshes the groups and their permissions after migrations (`ensure_roles`).

Role	Can see	Can change
Administrators (and superusers)	Everything	Everything, including users, imports and settings
Surveyors	Everything	Properties they manage, with their photos, leases, components, planned tasks, compliance records and jobs; new properties they add; shared contact details
Contributors	Everything	Notes, documents and correspondence on cases they are added to
Viewers	Everything	Nothing
Signed in with no role	Help and open data only	Nothing

Three layers enforce this: `RoleRequiredMiddleware` blocks anyone without a role; Django model permissions decide what kind of record a role may add or change; `can_edit(user, obj)` decides whether this user may change this particular record. The API applies the same rules through `PhaemaPermission`.

Audit trail

`apps/core/audit.py` connects to Django's save and delete signals for every business model. Each create, change and delete writes an `AuditEntry` with the time, the user (from `CurrentUserMiddleware`), the record and the old and new values. Users see it under Changes (`/changes/`); there is no screen for editing or deleting entries.

Access areas and cover

Roles decide what a person can change; **access areas** (`AccessArea`, `apps/core/access.py`) decide which optional modules they can open. Each area lists modules it opens in full, and may grant `own_patch_finance` (rent ledger and service

charges limited to the person's patch) and `finance_actions` (raise rent, credit notes, finance exports, service charge approvals and finance imports, for Surveyors). Five areas are created by migration `core.0013` and every user existing then was added to all of them.

Enforcement is on the server. Every view of an optional module goes through `access.module_view(module, off_template)`, which returns 404 if the council has the module switched off and 403 if the person's areas do not open it, and sets `request.access_scope` to `"full"` or `"patch"`. Templates receive `modules` as `ModulesForUser`, so every `modules.*_enabled` test is per person. The patch is computed by `patch_people()`: the person, everyone below them through `StaffProfile.supervisor` (the whole chain), and anyone they cover through a current `Cover` and that person's chain. The same rules apply to lease and party rent panels, dashboards, My work, the report builder (`DATASET_MODULES` in `apps/reports/views.py`), the change log (`audit_visible`), Hand over work receivers and works order approvers. Area membership and cover changes are written to the change log. Tests: `apps/core/test_access.py`; other tests put new users in every area unless marked `own_areas`.

Projects

`apps/projects` holds larger pieces of work with or without properties. A `Project` has a `ProjectType` (kind, starter phase names, confidential by default), a phase (`PHASE_ORDER`) and a status; `workflow.record_gateway()` is the only way to move phase: approved advances (gateway 4 closes), rejected stops. Visibility is in `rules.py`: `visible(user)` filters lists, reports (`_project_rows` in `apps/reports/views.py`) and property pages, so confidential projects reach only their team and administrators; `_project()` in the views returns 404 otherwise and writes a `ProjectAccess` row for each opening of a confidential project. Documents are stored under `projects/` and served only by `project_file`; `core/views.protected_file` refuses that folder.

The budget (`BudgetLine`, `BudgetYear`) is keyed by the starting year of the April–March financial year (`financial_year_of`). `can_see_budget` and `can_edit_budget` add finance staff through their access areas. `importing.SpendWorkbook` (Import, Project spending) loads actuals. Cases, jobs and valuations carry a nullable `project` foreign key; `project_for()` shows the link only to people who can see the project.

`completion.py` builds the checklist from delivery onwards. Steps link to admin add or change forms with GET parameters to prefill them; it never writes to the register. Manual steps are `CompletionStep` rows; automatic steps are computed (a linked property; `Property.disposed_on` set, which also removes the property from open data).

`site_from_inspire` unions INSPIRE parcels into `site_boundary` (EPSG:27700); `map.geojson` returns the site and properties in WGS 84 under the same visibility. Demo projects: `apps/projects/demo.py`, called by `load_demo_data`.

Site and buildings. `Property` holds the physical description of a site or building: location, description, when built, construction, storeys, roof, windows, parking, services and solar panels. The lists of choices are `TextChoices` on the model; `physical_rows()` returns the recorded facts as label and value pairs for the property page and reports. Site area is stored in hectares; the admin form (`PropertyForm`) also takes square metres and converts. `PropertyPlan` holds floor plans as PDF, DWG or DXF files up to 50 MB under `plans/`, checked by `validate_plan` in `apps/core/files.py` (extension and file signature) and deleted from storage with the record. `apps/properties/report.py` builds the property report, headed Initial valuation report when opened from a case in the Asset valuation group (`/properties/<id>/report/?case=<id>`). It writes the document library's markup, so `apps/documents/render.py` produces both the page and the Word file (`?format=docx`). It includes agreements that have not ended and open cases on the property, its parts and the properties it is part of, and ends with `gaps()`, the list of what is not yet recorded. Tests: `apps/properties/test_site_and_buildings.py`.

Site visit sheets. `apps/cases/visit.py` builds the sheet for a case that has a property (`/cases/<id>/site-visit/`, Word with `?format=docx`): the case, what to know before visiting (from `apps/compliance/safety.py`, plus the hazards of the buildings within and the current occupiers), the property information, and the checklist. It is read-only: nothing entered on it is saved. Two things are set for each case type. `CaseType.visit_parts` is the list of property parts to print; the parts are the named sections of `apps/properties/report.py` (`PARTS`, `sections()`), which the property report also uses. `CaseTypeChecklistItem` rows are the checklist: heading, item, guidance, kind of answer, an optional detail of the property record to print beside it (`checks`; `visit.recorded()` reads the value from the property, then what it is part of, then the parts within it) and `each_building`, which repeats the item under each building of a site. Starting values for the case types PHAEMA ships with are in `apps/cases/seed.py` and were applied to existing installations by migration `cases.0005`; a council's later changes are not overwritten. The phone and print layouts are plain CSS on `.phaema-sheet` in `static/css/phaema.css`. Tests: `apps/cases/test_site_visit.py`.

Drag and drop. Every file chooser in the templates is wrapped in the GOV.UK Frontend file upload component (`<div class="govuk-drop-zone" data-module="govuk-file-upload">`), which lets a file be dropped onto it and falls back to the plain chooser without JavaScript; no other script is involved. A test checks that every chooser is wrapped and has the label the component needs. Three places take several files at once through a `multiple` input named `files`: the case page (action `file_dropped`: `.eml` and `.msg` files go through the existing email filing, everything else becomes a `CaseDocument`), the

agreement page (`/leases/<id>/files/` , PDFs as `LeaseDocument` of the chosen type) and the property page (`/properties/<id>/files/` , floor plans or photos). Each file passes the same validators as a single upload and is filed on its own, so one refused file does not stop the others; `file_each()` in `apps/core/files.py` applies the limit of 20 files a time and collects the reasons, and `title_from_filename()` names each record. The pages submit as ordinary forms, so the existing permission checks and CSRF protection apply. Tests: `apps/core/test_drag_and_drop.py` .

PDF viewer. PDFs are shown inside PHAEMA with PDF.js, kept in `static/vendor/pdfjs/` (the `pdfjs-dist` package from npm: the library, its worker, the viewer components and their stylesheet and images, character maps, standard fonts and WebAssembly decoders; about 6 MB, no build step, nothing fetched from the internet). No link in the templates changed. The two views that serve files, `core/views.protected_file` and `projects/views.project_file` , ask `apps/core/viewer.py` whether the request is a person opening a page: a GET for a PDF whose `Accept` header contains `text/html` and which has no `raw` or `download` parameter. If so they return `templates/core/pdf_view.html` at that same address instead of the file; the page's script (`static/js/pdf-view.mjs`) then fetches the file from the same address with `?raw=1` . Everything else (the viewer's own request, the API, scripts, images, other file types) gets the file exactly as before, and `?download=1` sends it as an attachment. The same permission checks run before either answer, so the viewer opens nothing a person could not already open. Both answers carry `Vary: Accept` , and the page is `no-store` . `viewer.describe()` finds the record that holds the file by looking the stored path up in every `FileField` , and takes the title, the download file name (the original name where one was kept) and the Back link from it; a file no record holds gets its stored name and a link back to the referring page if that is on the same host. The page works without JavaScript by offering the file to open or download. Scripting inside PDFs, XFA forms and `eval` are off, and links in a PDF open in a new tab with `noopener noreferrer` . Password-protected or damaged files show a message and the download link. The strip of small pages is PHAEMA's own code in the same script (the PDF.js package has no thumbnail component): one button per page, each drawn to a small canvas only when it scrolls into view, one at a time, so long documents stay quick; whether the strip is open is remembered in the browser's local storage. To update PDF.js, follow `static/vendor/pdfjs/README.md` . Tests: `apps/core/test_pdf_viewer.py` .

Leavers and hand-overs

Administrators use **Hand over work** (`/people/hand-over/` , view `apps/core/handover_views.py` , logic `apps/core/handover.py`) to move a person's open work to colleagues and, optionally, switch off their sign-in in the same step. Each kind of work is an `Item` naming its query, the user field it changes and the roles that may receive

it: managed properties, open cases, open case tasks, pending case approvals, case contributions, open lease events, open maintenance jobs, pending works-order approvals, open valuation audit queries, projects led, project team places, open project tasks and risks, and supervised staff. Closed and completed records are never changed.

All moves run in one database transaction. Records are saved one at a time, not with a bulk `update()`, so each change fires the audit signals and appears in the change log under the administrator. Receivers are checked on the server: only active Surveyors and Administrators (and Contributors, for case contributions) are accepted. User accounts are deactivated, never deleted, so names stay on historical records. When adding a model with a user field, add an `Item` for it in `handover.items()`.

Files

- Uploaded photos and documents are stored outside the web root and served only through `/files/`, which needs sign-in.
- Files are written with permissions `0640`; form data other than files is limited to 5 MB per request.
- Responses carry `X-Content-Type-Options: nosniff`; only PDF, JPEG and PNG open in the browser, everything else downloads.

Web protections

- CSRF protection on every form (Django `CsrfViewMiddleware`).
- Clickjacking protection: `X-Frame-Options: DENY` in production.
- Template auto-escaping against cross-site scripting.
- The ORM parameterises all database queries.
- Excel and CSV exports neutralise cells starting `=`, `+`, `-` or `@`, so a spreadsheet cannot run formulas planted in data.
- The calendar feed address contains a long random token that acts as its password; users can reset it.

Set at the web server, not in PHAEMA

These should be configured on the reverse proxy in front of PHAEMA (see Deployment):

- HTTPS with TLS 1.2 or later, and HTTP redirected to HTTPS;
- `Strict-Transport-Security` (HSTS);
- limits on repeated sign-in attempts, or the council's web application firewall;
- a `Content-Security-Policy` allowing scripts and styles from the PHAEMA host only (and the OS or OpenStreetMap tile host for maps).

Installation and configuration

PHAEMA installs as three Docker containers with one command, `./phaema.sh start`, on any machine that runs Docker Compose, including Apple Silicon Macs.

Requirements

Item	Minimum for a test install	Suggested for one council in production
Processor	2 cores (x86-64 or ARM64)	4 cores
Memory	4 GB	8 GB
Disk	10 GB	50 GB plus document storage, on encrypted disks
Software	Docker Engine with Compose v2, Git, Bash	The same, on a supported Linux (Ubuntu 24.04 LTS or Debian 12)
Network	None beyond the machine	HTTPS reverse proxy, outbound SMTP for reminders, optional outbound HTTPS to the OS Maps API

The production figures are a starting estimate, not a measured benchmark.

The three containers

Service	Image	Does
`db`	`docker/db` (PostgreSQL 17 + PostGIS 3)	Holds all data in the `db-data` volume
`web`	`Dockerfile` (Python 3.12)	Runs migrations on start, then Gunicorn on port 8000, bound to `127.0.0.1` only
`scheduler`	Same image as `web`	Runs `daily_tasks` every hour

Uploaded files live in the `media` volume, shared by `web` and `scheduler`.

First install

1. Install Docker and Git (on a Mac: Homebrew, Colima and Docker, as in `docs/setup-mac.md`).
2. `git clone https://github.com/phaema/phaema-app.git && cd phaema-app`
3. `cp .env.example .env` and edit it (below). Never commit `.env`.

4. `./phaema.sh start` — builds the images and waits until `/health/` answers.
5. `./phaema.sh admin` — creates the first superuser.
6. Sign in, open Manage data, Module settings, and switch on the modules the council uses.
7. Optionally `./phaema.sh demo-data` to load the fictional demonstration estate.

Environment variables

Variable	Default	Set it to
`SECRET_KEY`	a development value	A long random string, unique to each install. Required in production
`DB_PASSWORD`	`phaema-local`	A strong database password
`DEBUG`	`False`	Always `False` outside development
`SECURE_COOKIES`	`False` in Compose	`True` whenever PHAEMA is reached over HTTPS
`ALLOWED_HOSTS`	`localhost,127.0.0.1`	The host names users type, comma separated
`PHAEMA_ORGANISATION_NAME`	Demonstration Council	The council's name for the header
`PHAEMA_BASE_URL`	`http://localhost:8000`	The public address, used in email links
`EMAIL_HOST`, `EMAIL_PORT`, `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD`	blank, 587	The council's SMTP relay. Blank writes emails to the log instead
`PHAEMA_PASSWORD_RESET_MINUTES`	60	How long a forgotten-password link works, in minutes
PHAEMA_SSO_ISSUER	blank	Address of the council's sign-in service for single sign-on. For Microsoft 365: https://login.microsoftonline.com/<tenant ID>/v2.0. Blank means single sign-on is off
PHAEMA_SSO_CLIENT_ID	blank	The application (client) ID the sign-in service gave PHAEMA
PHAEMA_SSO_CLIENT_SECRET	blank	The client secret. Note its expiry date: sign-in stops when it expires
PHAEMA_SSO_NAME	Microsoft	The name on the sign-in button: Sign in with Microsoft
`DEFAULT_FROM_EMAIL`	`PHAEMA <phaema@localhost>`	The sender address for reminders

Three more variables matter on a server behind an HTTPS proxy:

Variable	Default	Set it to
`BEHIND_HTTPS_PROXY`	`False`	`True` when a reverse proxy ends HTTPS, so Django trusts its `X-Forwarded-Proto` header. Safe because `web` listens on localhost only

Variable	Default	Set it to
`CSRF_TRUSTED_ORIGINS`	blank	Any extra `https://` addresses users reach PHAEMA by. An `https://` `PHAEMA_BASE_URL` is trusted automatically
`EMAIL_USE_TLS`	`True`	`False` only for a relay that does not support STARTTLS

A typical server `.env` therefore has `SECURE_COOKIES=True`, `BEHIND_HTTPS_PROXY=True`, `ALLOWED_HOSTS=phaema.example.gov.uk` and `PHAEMA_BASE_URL=https://phaema.example.gov.uk`.

Council-specific choices (modules, letter details, invoice prefix, approval limits, Ordnance Survey Data Hub key and map style, open data) are kept in the database under Module settings, not in environment variables.

Control script

Command	Does
`./phaema.sh start`	Build if needed, start, wait for health check
`./phaema.sh stop`	Stop; data is kept
`./phaema.sh update`	`git pull --ff-only`, rebuild and restart (migrations run automatically)
`./phaema.sh status` / `logs`	Show container state / follow logs
`./phaema.sh admin`	Create a superuser
`./phaema.sh demo-data [--reset]`	Load or reload the demonstration estate
`./phaema.sh import-inspire FILE`	Import HM Land Registry INSPIRE parcels
`./phaema.sh reminders`	Send due reminders now
`./phaema.sh backup`	Write a compressed database dump to `backups/`
`./phaema.sh reset-data`	Delete all data after typing DELETE

Deployment and operations

In production PHAEMA runs on one UK-hosted Linux server or virtual machine per council, behind an HTTPS reverse proxy, with nightly database backups copied off the server.

Hosting pattern

1. A Linux server in a UK data centre (council-owned, or a supplier such as IONOS UK cloud).
2. Docker and the PHAEMA stack as in Installation, with `.env` set for production.
3. A reverse proxy (Caddy, nginx or the council's load balancer) that ends HTTPS and forwards to `127.0.0.1:8000`. The `web` port is bound to localhost so it is never exposed directly.
4. A DNS name such as `phaema.example.gov.uk`, added to `ALLOWED_HOSTS` and `PHAEMA_BASE_URL`.
5. Outbound SMTP to the council's mail relay or Microsoft 365 for reminders.

One stack per council keeps each council's data in its own database and makes it simple to show where the data lives. Several councils can share one physical host, each in its own stack with its own volumes and secrets.

Backups and restore

- `./phaema.sh backup` writes `backups/phaema-YYYYMMDD-HHMMSS.sql.gz` with `pg_dump`. Schedule it nightly with cron and copy the file to separate UK storage.
- Back up the `media` volume (photos and documents) on the same schedule, for example with `docker run --rm -v phaema-app_media:/m -v $PWD/backups:/b alpine tar czf /b/media.tgz -C /m .`.
- To restore: start an empty stack, then `gunzip -c FILE.sql.gz | docker compose exec -T db psql -U phaema phaema`, and unpack the media archive into the volume.
- Test a restore at least every quarter.

Upgrades

`./phaema.sh update` pulls the latest code, rebuilds and restarts. Migrations run automatically when `web` starts. Take a backup first. To roll back, check out the previous version with Git (releases are not tagged yet) and restore that day's backup, because migrations are not reversed automatically.

Monitoring

- `/health/` returns success when the app and database answer. Point an uptime monitor at it.
- `./phaema.sh logs` shows Gunicorn, Django and scheduler output. On a server, ship Docker logs to the council's log service.

- The `scheduler` container logs each hourly `daily_tasks` run; a missing run means reminders are not going out.
- All containers restart automatically (`restart: unless-stopped`).

Scaling

One server with 3 Gunicorn workers suits a single council's estates team. For more users, raise `--workers` in the Dockerfile command, or move PostgreSQL to a managed UK database service and run more `web` containers behind the proxy, with `media` on shared storage.

Testing and quality

PHAEMA has 391 automated tests in 37 files, run on every pull request by GitHub Actions against a real PostGIS database; a pull request is merged only when they pass.

Releases are numbered and published on GitHub. The first is 0.9.0, the last alpha, published on 10 November 2026; beta releases will be numbered from 1.0.0-beta.1 and the first version for live council use will be 1.0.0. The number is held once, as `PHAEMA_VERSION` in `phaema/settings.py`, and is shown in the API description. It is changed in the pull request that precedes each release.

Running the tests

```
pip install -r requirements.txt
export DATABASE_URL=postgis://phaema:phaema@localhost:5432/phaema
python -m pytest
```

The database user needs permission to create the test database. A full run takes a few minutes. `pytest.ini` points pytest at `phaema.settings` and collects `tests.py` and `test_*.py`.

What the tests cover

Area	Test files	Tests
Properties, maps, INSPIRE, open data	`properties/tests.py`, `test_week2.py`, `test_features.py`	38
Leases, events, parties, reminders	`leases/tests.py`, `test_week3.py`, `test_events_parties.py`	28

Area	Test files	Tests
Cases and professional standards	`cases/test_cases.py`, `test_standards.py`	29
Service charges	`servicecharges/test_service_charges.py`	16
Reports and exports	`reports/test_reports.py`	13
Rent ledger	`ledger/test_ledger.py`	13
Roles, My work, place filters	`core/test_roles.py`, `test_mywork.py`, `test_places.py`	14
Maintenance	`maintenance/test_maintenance.py`	8
Imports	`imports/test_imports.py`	8
Compliance	`compliance/test_compliance.py`	8
Valuations	`valuations/test_valuations.py`	7
Projects	`projects/test_projects.py`	20
Document templates	`documents/test_documents.py`	7
Help pages	`help/test_help.py`	6

Other checks

- **Migrations:** CI runs `makemigrations --check --dry-run`, so a model change without its migration fails the build.
- **Lint:** `python -m pyflakes apps phaema conftest.py` before each pull request.
- **Accessibility:** axe-core runs in a headless browser over the main pages during development; manual keyboard and screen reader checks for new screens.
- **User acceptance:** the PHAEMA Test Programme document gives step-by-step tests for each section, against the demonstration data.
- **Demo data:** `load_demo_data` is itself tested, so record counts in the training and test programme stay true.

Integrations and data exchange

Today PHAEMA exchanges data through documented files (Excel, CSV, GeoJSON, iCalendar, Word) and an open REST API. Live connectors to finance, legal and CRM systems are planned for later releases.

Imports

Import	Where	Format
Properties	Properties, Import (`/properties/import/`)	Excel template
Parties; leases and licences (with breaks)	Manage data, Import (`/import/`)	Excel workbook, one sheet per record type
Rent receipts from the finance system	Import hub	Excel; matched to leases by `PHC` invoice number or reference
Service charge spending	Import hub	Excel
Contractors and components	Import hub	Excel
Valuer's returned figures	Import hub	Excel
HM Land Registry INSPIRE Index Polygons	Properties, INSPIRE import, or `./phaema.sh import-inspire`	`.zip` or `.gml` as downloaded from HM Land Registry

Each Excel import has a downloadable template. Every row on every sheet is checked first; nothing is saved unless the whole file is clean, and then everything is saved in one database transaction.

Exports

Export	Format	For
Any list or report	Excel or CSV	General use; formula injection neutralised
Finance export	Excel or CSV, 21 fixed columns	Invoice or journal import into Unit4, Oracle, SAP, Civica or others. Each export is a numbered batch so nothing is sent twice
Service charge budgets and accounts	Excel or CSV	Tenants, auditors

Export	Format	For
Valuation schedule, audit pack	Excel	Finance and external audit
Letters and notices	Word (`.docx`)	Legal and correspondence
Open data: land and assets; property available	CSV, JSON, GeoJSON at `/open-data/`	Local Government Transparency Code publication, without sign-in
Lease event calendar	iCalendar feed `/calendar/<token>.ics`	Outlook, Google Calendar, Apple Calendar

The finance export (`apps/ledger/finance.py`) has two columns added at the end, Billing contact and Billing email, so existing column mappings are not disturbed. Its customer address is the agreement's billing address where one is set.

REST API

- Base address `/api/` , with an OpenAPI 3 schema at `/api/schema/` and interactive documentation at `/api/docs/` .
- Resources: `properties` , `titles` , `leases` , `parties` . Results are paged 50 at a time.
- Authentication by session or HTTP Basic; permissions follow the user's PHAEMA role (see Security).
- Client libraries and connectors are to be licensed Apache-2.0 so suppliers can build on them freely.

External services

Service	Used for	Needed?
Ordnance Survey Maps API	Base maps, with the council's OS Data Hub key (free under the Public Sector Geospatial Agreement)	Optional; OpenStreetMap is used without a key
SMTP relay	Reminder and case emails	Optional; without it emails go to the log
HM Land Registry	INSPIRE polygons, downloaded by the council	Optional

PHAEMA bundles no licensed data. SFG20 schedules, OS premium data and Land Registry services are connected under each council's own licence.

Extending PHAEMA and contributing

New features are added as Django code in the matching app, with tests, translatable text and an audit registration, and contributed back under AGPL-3.0 with a Developer Certificate of Origin sign-off.

Conventions

- Every source file starts `# SPDX-License-Identifier: AGPL-3.0-or-later`.
- Python follows PEP 8; keep functions small. Business rules live in plain modules beside the models (for example `cases/standards.py`, `compliance/safety.py`, `core/mywork.py`), not in templates.
- All user-facing text is wrapped in `gettext` or `{% translate %}` so a Welsh translation can be added under `locale/`.
- Screens follow GOV.UK Design System patterns and the GOV.UK style guide's plain English.
- Server-rendered pages must work without JavaScript.

Adding a feature: checklist

1. Add or change models in the right app and run `python manage.py makemigrations`.
2. Register new business models with the audit trail in the app's `apps.py` (`audit.register(model)`).
3. Add permissions to the right roles in `core/roles.py`, and record-level rules to `can_edit` if surveyors may change it.
4. Add views, URLs and templates using the shared partials in `templates/core/`.
5. Offer Excel and CSV export for any new list (`reports/export.py`).
6. Write tests, including the role rules.
7. Update the help page in `apps/help/pages/` and, if screens changed, the training course in `training/course.py`.
8. Extend `load_demo_data` with fictional examples (ZZ postcodes, 9999 UPRNs, ZZ title numbers).
9. Run the tests and pyflakes, then open a pull request.

Licensing

Part	Licence
The application	AGPL-3.0-or-later: anyone running a changed version as an online service must publish their changes
Connectors, client libraries, SDKs	Apache-2.0
Data model and schema documentation	CC BY 4.0
Data councils publish through PHAEMA	Open Government Licence v3.0

Councils commissioning features should require the supplier to release the work under the project licence. Security problems should be reported privately, not in public issues to stuart@phaema.co.uk, as `SECURITY.md` in the repository explains, with response times and the known security gaps.

Compliance mapping

This table shows how PHAEMA's design supports each point of the Technology Code of Practice and the main legal duties. It is a self-assessment for councils to check, not a certification.

Technology Code of Practice

Point	How PHAEMA supports it	Council still needs to
1. Define user needs	Designed with a chartered surveyor around estates tasks; role dashboards; test programme	Run its own user research
2. Make things accessible and inclusive	GOV.UK Frontend, WCAG 2.2 AA target, themes, text sizes, keyboard use, axe checks	Commission an audit and publish an accessibility statement
3. Be open and use open source	AGPL source on GitHub; all dependencies open source	Publish any changes it makes

Point	How PHAEMA supports it	Council still needs to
4. Make use of open standards	UPRN, British National Grid, GeoJSON, CSV, OpenAPI 3, iCalendar, `.docx`, `.xlsx`	Use UPRNs in its own records
5. Use cloud first	Containerised; runs on any UK cloud or on-premises	Choose a UK host
6. Make things secure	Role-based access, audit trail, secure cookies, CSRF, protected files	Run HTTPS, patching, penetration test, IT health check
7. Make privacy integral	Minimal personal data (tenants' contact details); data stays in the council's own database	Complete a DPIA and record of processing
8. Share, reuse and collaborate	One code base for every council; contributions return to all	Contribute back commissioned work
9. Integrate and adapt technology	REST API, documented finance export, spreadsheet imports	Map the finance export once
10. Make better use of data	Report builder, exports, Transparency Code open data	Publish its open data
11. Define your purchasing strategy	No licence fee; hosting and support can be bought from any supplier	Procure hosting and support competitively
12. Make your technology sustainable	Small server footprint, no client build, server-rendered pages	Choose an energy-efficient host
13. Meet the Service Standard	Internal staff tool; follows GOV.UK patterns	Decide whether a service assessment applies

Legal and other standards

Duty or standard	PHAEMA's part
Public Sector Bodies Accessibility Regulations 2018	WCAG 2.2 AA target and accessible components; statement to be published by the council

Duty or standard	PHAEMA's part
UK GDPR and Data Protection Act 2018	Role-based access, audit of every change, UK hosting, export and deletion through the admin; the council is the controller
Local Government Transparency Code 2015	Land and assets dataset in CSV, JSON and GeoJSON
NCSC Cloud Security Principles	Supported by the hosting pattern: separate stack per council, encrypted transport, backups; the host supplier must evidence the rest
Building Safety Act 2022 (Golden Thread)	Building safety records, tiers and superseded documents kept with history
CIPFA Code of Practice on Local Authority Accounting	Asset classes, valuation bases and year-end schedules for finance
RICS professional standards	Red Book valuation fields, conflict of interest checks, MEES and dilapidations prompts

Troubleshooting and reference

Most problems come from three settings: `ALLOWED_HOSTS`, HTTPS cookie and CSRF settings, and email.

Common problems

Symptom	Likely cause	Fix
"Bad Request (400)" on every page	The host name is not in `ALLOWED_HOSTS`	Add it to `.env`, then `./phaema.sh start`
Sign-in returns "Forbidden (403) CSRF verification failed" behind HTTPS	The proxy setting is off, so Django sees plain HTTP and the browser's `https://` origin does not match	Set `BEHIND_HTTPS_PROXY=True` and an `https://` `PHAEMA_BASE_URL`, then `./phaema.sh start`
Sign-in works over HTTP but the session is lost	`SECURE_COOKIES=True` without HTTPS	Use HTTPS, or `False` only for local tests
"Waiting for PHAEMA to start" never finishes	Database still starting, or a migration failed	`./phaema.sh logs`, look for the error in `web`

Symptom	Likely cause	Fix
`docker: command not found` or cannot connect on a Mac	Colima not running	`colima start`
Port 8000 already in use	Another program on 8000	Stop it, or change the left-hand port in `docker-compose.yml`
Reminder emails never arrive	`EMAIL_HOST` blank, so emails go to the log	Set the SMTP variables; check `scheduler` logs
Demo data shows old counts	Old code still running	`./phaema.sh update`, then `./phaema.sh demo-data --reset`
"Missing staticfiles manifest" outside Docker with `DEBUG=False`	No `collectstatic` manifest	Run `python manage.py collectstatic`, or use `DEBUG=True` for the development server
Map shows OpenStreetMap, not OS	No OS key in Module settings	Enter the council's OS Data Hub key

Known issues to fix before production

Issue	Effect	Planned fix
No limit on repeated sign-in attempts	Password guessing is not slowed	Rate-limit at the proxy now; add in-app throttling later
Single sign-on not yet proved on a live council tenant	It is tested against a stand-in sign-in service only; a council's own conditional access rules have not been tried	Try it with a pilot council's Microsoft 365
No chat, file store or GIS connectors (Microsoft 365 or open-source alternatives)	Reminders and approvals arrive by email only; key dates are not in Outlook; documents are stored in PHAEMA, not SharePoint	On the roadmap: an optional connectors module on open standards, off by default

Issue	Effect	Planned fix
No tagged releases	No version numbers or security support periods (SECURITY.md now gives the security contact and response times)	Publish with the first release
No formal accessibility audit	Statement cannot yet be published	Commission an audit

Fixed in November 2026: sign-in and forms failing behind an HTTPS proxy (`BEHIND_HTTPS_PROXY`, and `CSRF_TRUSTED_ORIGINS` and `EMAIL_USE_TLS` now passed through Docker Compose).

Useful addresses

Address	Purpose
`/health/`	Health check, JSON `{"status": "ok"}`
`/admin/`	Django administration (superusers and administrators)
`/api/docs/`	Interactive API documentation
`/changes/`	Audit trail
`/help/`	In-app help and training courses
`/open-data/`	Published datasets

Administrators also have `/people/hand-over/` to move a leaver's work to colleagues. Administrators and supervisors have /people/cover/ to arrange cover for absence.